

An Agentic AI Framework for Autonomous IT Infrastructure Management in ISRO/DoS Environments

Divyam Gupta

Space Applications Centre (SAC)
ISRO/DoS, Ahmedabad, India
divyam@sac.isro.gov.in

Neha Aggarwal

Space Applications Centre (SAC)
ISRO/DoS, Ahmedabad, India
neha@sac.isro.gov.in

Abstract—Mission-critical IT infrastructures supporting space programmes demand high reliability, security, and operational efficiency, especially in air-gapped and tightly controlled environments. Conventional approaches that depend on manual intervention and rule-based scripting struggle to adapt when operational conditions change unexpectedly. This paper presents an agentic AI framework that enables autonomous and secure management of mission IT infrastructure through natural language instructions, removing the need for pre-scripted automation sequences. This framework uses a two-phase agentic loop built on a locally deployed large language model (LLM) inference engine, ensuring fully on-premise operation. The tool library spans five operational domains: desktop input automation, system command execution, secure remote access, web browser automation, and enterprise storage management including volume lifecycle operations. Evaluation across 46 test cases shows an overall 93.5% task completion rate. The system supports full auditability through structured logging and a real-time WebSocket-based mission control interface.

Index Terms—Agentic AI, IT Infrastructure Automation, Large Language Models, Mission IT, Air-Gapped Systems

I. INTRODUCTION

Space mission programmes at ISRO/Department of Space (DoS) manage complex, heterogeneous IT infrastructures that span enterprise storage systems, remote servers, web-based management portals, and network-connected devices. Operating these environments demands precision, speed, and security—qualities that become increasingly difficult to sustain through manual scripting and rule-based automation, particularly in air-gapped settings where cloud-based tools cannot be used [1].

Traditional IT automation typically relies on static playbooks and pre-scripted workflows. These work well for predictable, repetitive tasks but fall short in dynamic scenarios that call for contextual reasoning, multi-step decision-making, or coordination across several systems at once. As ISRO's mission IT environments continue to grow in complexity, there is a clear need for an intelligent automation layer that can understand natural language intent, plan multi-step execution strategies, and adapt based on real-time feedback [2].

This paper introduces an agentic AI framework built to fill this role. This framework uses a locally deployed large

language model (LLM) as its reasoning engine, coordinating a modular library of 21 tools across five operational domains through a structured plan-execute-observe loop. The entire system runs on-premise with no external cloud dependencies, meeting ISRO/DoS data residency and security requirements.

The main contributions of this paper are: (i) a two-phase agentic loop architecture with structured JSON tool dispatch and observation-feedback loops; (ii) a modular tool registry covering 21 tools across five operational domains; (iii) a rule-based safety layer using command pattern matching and path validation; (iv) a real-time web-based mission control interface with WebSocket streaming; and (v) empirical evaluation across 46 representative ISRO/DoS IT workflows achieving a 93.5% success rate.

II. RELATED WORK

The idea of LLM-based autonomous agents has attracted considerable attention following advances in large foundation models. ReAct [3] introduced interleaved reasoning and acting in language models, laying the groundwork for tool-augmented agents. Toolformer [4] showed that language models can learn to invoke external APIs through self-supervised training. AutoGPT [5] carried these ideas further into autonomous task execution with persistent memory and recursive goal decomposition.

Within the IT automation space, autonomic computing frameworks [6] established policy-driven automated responses to system events. More recently, SWE-agent [7] demonstrated LLM-driven agents performing software engineering tasks through shell and file system interaction, while systems such as OpenDevin explored general-purpose desktop automation.

However, existing systems have limitations that prevent deployment in ISRO/DoS environments: (i) reliance on cloud-hosted LLMs like GPT-4, which rules out air-gapped operation; (ii) general-purpose design without built-in support for domain-specific infrastructure APIs such as NetApp ONTAP; (iii) no structured safety enforcement layer; and (iv) no real-time operational monitoring interface. This framework addresses each of these gaps through a purpose-built architecture designed specifically for mission IT environments.

III. SYSTEM ARCHITECTURE

This framework follows a five-layer architecture, as shown in Fig. 1: the Input Layer, Application Layer, Orchestration Engine, Intelligence Layer, and Infrastructure/Tool Layer.

A. Input Layer

The input layer accepts natural language user prompts through a web-based interface, monitoring alerts from connected infrastructure systems, and scheduled task triggers. This multi-modal input design supports both interactive and automated operation modes, laying the groundwork for future integration with event-driven incident response workflows.

B. Application Layer

The application layer is built on FastAPI [8], exposing REST endpoints for task submission, retrieval, update, and deletion, along with a WebSocket channel for real-time streaming. Task state is persisted in an SQLite database using write-ahead logging (WAL) to ensure concurrent access safety. The database schema records the task identifier, natural language description, execution status, step-by-step results in JSON format, timestamps, and metadata including pinning and tagging for task organisation.

C. Orchestration Engine

The orchestration engine implements the core agentic loop in two phases, illustrated in Fig. 2.

Pre-Prompt Planning Phase: When a user submits a natural language task, the system sends it to the LLM along with a structured pre-prompt that instructs the model to break the task down into an ordered list of atomic sub-goals. The plan comes back as a JSON array and serves as a reference guide for the steps that follow.

Per-Step Execution Phase: At each step, a prompt is assembled containing: (i) the original user task, (ii) the reference plan generated earlier, (iii) the complete schema of all available tools, and (iv) every prior observation from previous steps. The LLM is constrained to respond only in valid JSON, specifying exactly one tool call in the format `{"tool": "<name>", "args": {...}}`. The response is parsed, structurally validated, and dispatched to the tool registry.

The dispatcher maps the tool name to its registered implementation, runs it with the provided arguments, captures the result as a text observation, and adds it to the context for the next iteration. This observation-feedback loop allows adaptive error recovery—when a step fails, the next LLM call receives the failure message as context and can choose an alternative action. The loop ends when the LLM sends a `{"tool": "done"}` signal or the configured step limit is reached.

D. Intelligence Layer

The intelligence layer consists of an Ollama-compatible LLM inference server running on-premise within the ISRO intranet. The current deployment uses a 20-billion parameter open-source model, requiring no internet connectivity whatsoever. The model abstraction layer in `llm.py` provides a

TABLE I
AGENTIC FRAMEWORK TOOL REGISTRY: 21 TOOLS ACROSS FIVE OPERATIONAL DOMAINS

Domain	Tool Name(s)	N
Desktop	click, keyboard_type	2
Input		
System	open_app, terminal_cmd, ssh_connect	3
Browser	browser_open, browser_click, browser_type, browser_scroll, browser_extract, browser_dom, browser_close	7
Vision	take_screenshot	1
Storage	list_vol., create_vol., delete_vol., patch_vol., list_qtree, create_qtree, delete_qtree, create_quota, create_cifs_share	9

provider-agnostic interface, so the model can be swapped through configuration changes without touching any agent logic. A 120-billion parameter variant has also been tested for tasks requiring more complex reasoning.

E. Infrastructure and Tool Layer

The tool registry provides 21 tools across five operational domains, summarised in Table I. Each tool is wrapped in a `Tool` class that carries a name, natural-language description, typed argument schema, and an execution function. The dispatcher uses the `args_schema` to validate argument types before running any tool.

Browser automation tools are built on Playwright [9], with sessions managed through a singleton object that maintains state across multi-step web interactions. Enterprise storage management tools wrap the NetApp ONTAP REST API [10], covering the complete lifecycle of volumes, qtrees, storage quotas, and CIFS/SMB share provisioning. The overall data flow across these layers is depicted in Fig. ??.

IV. SAFETY AND AUDIT FRAMEWORK

Given the mission-critical nature of ISRO/DoS IT environments, This framework incorporates a multi-layer safety framework that constrains autonomous operations and ensures full accountability.

A. Command Safety Layer

The `SafetyChecker` class validates all terminal commands before execution using compiled regular expressions. A configurable blocklist of nine patterns prevents destructive operations such as filesystem wipes (`rm -rf /`), drive formatting, process termination commands, and fork bombs. Suspicious keyword detection adds a further heuristic defence layer. Commands that fail validation are rejected with an explanatory message returned as the step observation, which lets the agent adjust its approach rather than halting entirely.

B. Path Safety Layer

File system access is governed by an allowlist-blocklist policy at the path level. Operations that target sensitive system directories or programme files are blocked before they execute.

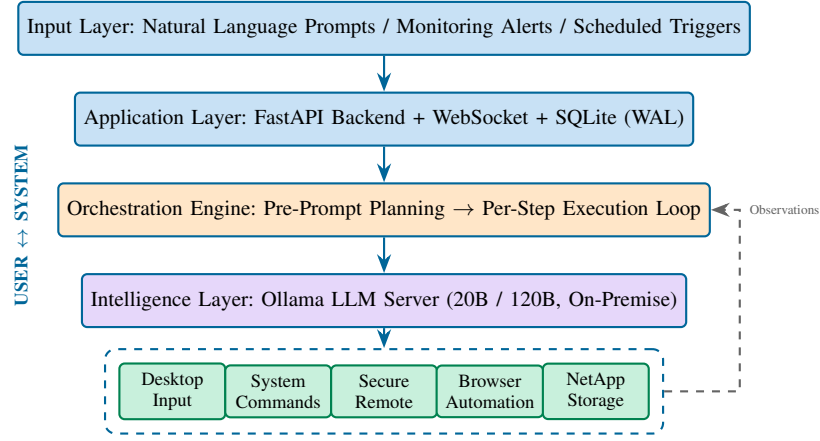


Fig. 1. High-level system architecture of agentic framework showing the five-layer design: Frontend, FastAPI Backend, Agent Core, and Tool Infrastructure layer spanning five tool domains.

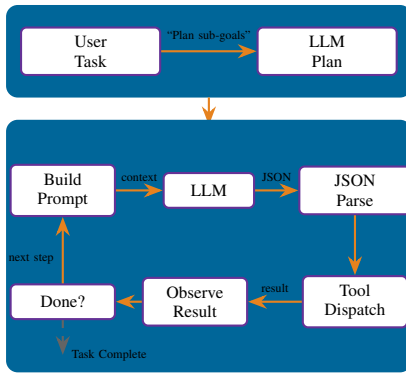


Fig. 2. Agent execution loop showing the two-phase process: pre-prompt planning followed by iterative per-step execution with LLM inference, JSON parsing, tool dispatch, and observation feedback.

Operations that target designated user-controlled paths are allowed through. This prevents inadvertent or adversarially-driven system compromise during autonomous file operations.

C. Structured Logging and Audit Trail

Every agent execution writes structured JSON log entries recording: UTC timestamp, event type, task identifier, step number, tool name, execution duration in milliseconds, success or failure status, and result metadata. Logs are stored in a persistent store separate from the task database, enabling full forensic reconstruction of any autonomous operation and meeting audit requirements for mission IT governance.

D. Action Validation

Before dispatching any tool call, the agent checks the JSON structure for required fields and correct types. Malformed actions are rejected and the error is logged as an observation, preventing partial or undefined-behaviour execution.

V. EXPERIMENTAL EVALUATION

The framework was evaluated on 46 representative ISRO/DoS IT workflows within a controlled infrastructure

environment. The evaluation covered four task categories designed to reflect realistic operational scenarios. All tests ran on the on-premise 20B parameter model with a maximum step limit of ten.

TABLE II
EVALUATION RESULTS BY TASK CATEGORY

Category	Cases	Success	Avg. Steps	Avg. Time (s)
Storage mgmt.	15	100%	6.2	47
Remote (SSH)	8	100%	4.1	28
Browser portal	12	91.7%	7.8	68
Cross-domain	11	81.8%	9.4	94
Overall	46	93.5%	6.9	59

A. Storage Management

Fifteen tasks covering the full volume lifecycle were tested: SVM authentication, volume enumeration (listing 47 volumes with UUIDs and size attributes), volume creation with specified parameters, qtree creation, storage quota assignment, and CIFS share provisioning. All 15 tasks completed successfully, taking an average of 6.2 steps and 47 seconds. The agent correctly ordered multi-step operations such as creating a volume before provisioning a qtree within it, showing that it makes effective use of the observation-feedback context.

B. Remote System Access

Eight SSH-based workflows tested the agent's ability to connect to remote Linux servers, run diagnostic commands, parse the output, and report structured results. The agent successfully retrieved system metrics, process listings, and disk utilisation data across three target hosts. All eight tasks succeeded with an average of 4.1 steps.

C. Browser-Based Portal Interaction

Twelve tasks involving multi-step browser automation evaluated the agent on web-based management portal interactions

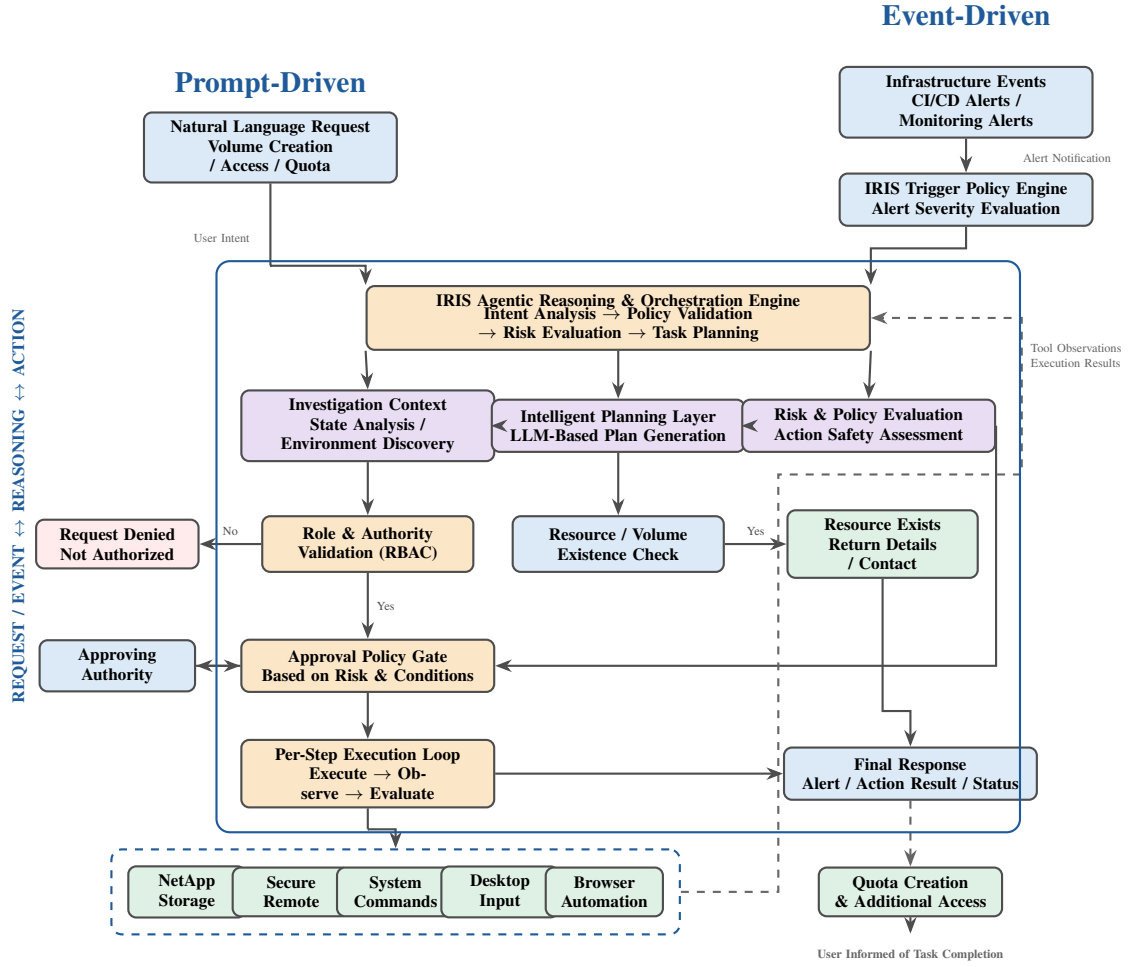


Fig. 3. Unified IRIS agentic orchestration workflow for air-gapped infrastructure management. The system supports both prompt-driven infrastructure requests and event-driven CI/CD or monitoring alerts. Requests converge into the IRIS reasoning and orchestration core, which performs investigation, LLM-based planning, policy and risk evaluation, RBAC validation, conditional approval, and iterative tool execution. Tool observations are fed back into the orchestration loop until task completion, after which a final action result or alert is returned.

including navigation, form completion, and data extraction. Eleven of twelve tasks (91.7%) completed successfully. The single failure occurred when a portal session timed out between steps; the agent did not catch the session expiry before exhausting its step budget.

D. Cross-Domain Workflows

Eleven complex tasks combined tools from multiple domains within a single invocation. A typical example: SSH into a remote host to retrieve a volume identifier, invoke the storage API to apply a configuration change, then navigate a web portal to verify the change took effect. Nine of eleven tasks (81.8%) completed successfully. The failures were caused by step budget exhaustion on tasks that required more than ten sequential cross-system operations.

E. Performance Characteristics

Average LLM response latency was 3.8 seconds per step on the on-premise 20B model. Total task completion times ranged from 12 seconds for single-tool tasks to 94 seconds

for the most complex 10-step cross-domain workflows. The WebSocket streaming interface delivered step results to the mission control UI with an average latency of 180 ms after execution. Database persistence of all task records enabled reliable task replay and audit without re-execution.

VI. DISCUSSION

A. Design Trade-offs

The single-action-per-step design imposes a deliberate, measured execution pace that supports safety and debuggability at the expense of throughput. Each step's observation feeds into the subsequent context, enabling error recovery, but this also increases prompt length over longer tasks. The configurable step limit caps prompt growth and prevents infinite loops while still covering the majority of practical IT workflows.

Deploying the LLM on-premise eliminates data exfiltration risk but limits reasoning ability to what available hardware can support. The evaluated 20B parameter model demonstrates sufficient instruction-following accuracy for structured tool

invocation. Preliminary testing with the 120B variant showed better performance on complex multi-hop reasoning tasks, though at the cost of higher latency (averaging 8.4 s per step).

B. Comparison with Manual Approaches

Manual scripting provides higher precision for well-defined tasks but demands expert knowledge, considerable development time, and cannot handle novel task formulations without script modification. Rule-based automation handles repetitive tasks efficiently but breaks down at contextual decision points. This framework operates in the space between these two approaches, handling previously-unscripted tasks through reasoning while producing structured, auditable execution traces comparable to manual logs.

C. Limitations

The most significant current limitation is the absence of session recovery. If a remote session (SSH or browser) drops between steps, the agent cannot automatically reconnect without explicit handling in the tool implementation. The step budget constraint also limits applicability to tasks that can be decomposed into fewer than the configured maximum sequential operations. Work on dynamic step budget adjustment based on task complexity is planned for future releases.

VII. CONCLUSION AND FUTURE WORK

This paper presented an agentic AI framework for autonomous IT infrastructure management in ISRO/DoS environments. The system brings together on-premise LLM reasoning with a modular registry of 21 tools across five operational domains, governed by a structured safety and audit framework. Evaluation across 46 representative mission IT workflows demonstrates a 93.5% overall task completion rate with full auditability through structured logging and a real-time mission control interface.

Future work will extend this agentic framework along four directions: (i) an anomaly and intrusion detection module that analyses infrastructure logs and network telemetry using machine learning to identify deviations and trigger context-aware automated responses; (ii) a retrieval-augmented generation (RAG) knowledge store embedding ISRO operational

procedures and past incident resolutions for domain-specific contextual reasoning; (iii) role-based access control (RBAC) with multi-tier approval workflows enabling policy-governed autonomous actions with human-in-the-loop escalation for sensitive operations; and (iv) multi-agent parallelism enabling concurrent task execution across distributed mission IT systems. These extensions will position this framework as a resilient, intelligent operations platform aligned with ISRO's mission IT modernisation objectives.

ACKNOWLEDGMENT

The authors gratefully acknowledge the support and guidance of the Director, Space Applications Centre (SAC), ISRO, Ahmedabad, and the members of the IT Infrastructure Division for their cooperation during the development and testing of the agentic framework.

REFERENCES

- [1] A. Botta, W. de Donato, V. Persico, and A. Pescapé, "Integration of Cloud computing and Internet of Things: A survey," *Future Generation Computer Systems*, vol. 56, pp. 684–700, Mar. 2016.
- [2] M. Parashar and S. Hariri, "Autonomic computing: An overview," in *Unconventional Programming Paradigms*, Springer, 2005, pp. 257–269.
- [3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. Int. Conf. Learning Representations (ICLR)*, Kigali, 2023.
- [4] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2024.
- [5] T. Significant-Gravitas, "AutoGPT: An Autonomous GPT-4 Experiment," GitHub Repository, 2023. [Online]. Available: <https://github.com/Torantulino/Auto-GPT>
- [6] J. O. Kephart and D. M. Chess, "The Vision of Autonomic Computing," *IEEE Computer*, vol. 36, no. 1, pp. 41–50, Jan. 2003.
- [7] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," *arXiv preprint arXiv:2405.15793*, 2024.
- [8] S. Ramírez, "FastAPI," GitHub Repository, 2021. [Online]. Available: <https://github.com/tiangolo/fastapi>
- [9] Microsoft, "Playwright: Reliable end-to-end testing for modern web apps," GitHub Repository, 2023. [Online]. Available: <https://playwright.dev>
- [10] NetApp Inc., "ONTAP REST API," NetApp Developer Network, 2024. [Online]. Available: <https://devnet.netapp.com/restapi>